

AI Web Tester — Development Work Summary

Claude + Playwright · FastAPI + Next.js · Local QA Automation Tool

📅 Date: May 19, 2026

🏠 Project: ai-web-tester

👤 Developer: Biswa

🤖 AI Pair: Claude Sonnet 4.6

✅ Features Shipped: 2 major · 1 bug fix

TABLE OF CONTENTS

1. Feature 1 — File Attachment Support (PDF, DOCX, Excel, TXT)
2. Feature 2 — Headed Browser Mode with Real-Time Execution Console
3. Bug Fix — "Not Found" Error in Headed Mode
4. Files Changed Summary
5. Architecture Overview

1

File Attachment Support

Attach PDF, DOCX, Excel, TXT files as test instructions for the AI agent

Goal: Allow users to attach structured documents (test plans, workflows, checklists) so the AI agent reads and follows them instead of relying only on a typed goal.

frontend/pages/index.js

- Added "Attach file" button next to Test Goal label
- Hidden `<input type="file">` with `fileInputRef`
- Accepts: `.pdf .doc .docx .xls .xlsx .txt .csv .md`
- Green removable file chips below textarea
- Deduplication by filename on multi-select
- Form submission switched from JSON → FormData

backend/main.py

- `/test` endpoint changed to `multipart/form-data`
- New `extract_file_text()` async helper
- Appends extracted text under "Instructions from attached files:"
- Credentials now passed as JSON string in form field
- Graceful fallback if file parsing fails

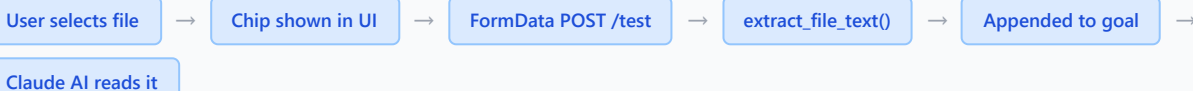
backend/requirements.txt

- **pdfplumber==0.11.4** — PDF text extraction
- **python-docx==1.1.2** — DOCX paragraph parsing
- **openpyxl==3.1.5** — Excel `.xlsx` row extraction
- `python-multipart` already present ✓

File Parsing Logic

- `.txt / .csv / .md` → UTF-8 decode
- `.pdf` → pdfplumber page-by-page text
- `.docx` → python-docx paragraph join
- `.xlsx` → openpyxl tab-separated rows
- All wrapped in `try/except` with warning log

```
# backend/main.py - extract_file_text helper
async def extract_file_text(file: UploadFile) -> str:
    content = await file.read()
    name = (file.filename or "").lower()
    if name.endswith((".txt", ".csv", ".md")):
        return content.decode("utf-8", errors="replace")
    if name.endswith(".pdf"):
        import pdfplumber
        with pdfplumber.open(io.BytesIO(content)) as pdf:
            return " ".join(page.extract_text() or "" for page in pdf.pages)
    if name.endswith(".docx"):
        from docx import Document
        doc = Document(io.BytesIO(content))
        return " ".join(p.text for p in doc.paragraphs)
```



Goal: Let developers see exactly what the AI is doing in the browser during a test — page navigations, clicks, typing, and scrolling — with a live console showing every action as it happens.

-  **Frontend Changes (index.js)**
- Headed / Headless toggle switch
 - Live status badge (● HEADED / ○ HEADLESS)
 - Slow Motion range slider (0–2000ms)
 - Dark terminal-style Execution Console panel
 - macOS traffic-light dots + LIVE pulsing indicator
 - Color-coded log lines (blue/yellow/green/red)
 - AI reasoning shown in dimmer text per step
 - Auto-scroll to latest console entry
 - Streaming via fetch() ReadableStream (not EventSource)
 - runTestStreaming() + handleStreamEvent() added

-  **Backend Changes**
- **browser.py** — slow_mo param → chromium.launch()
 - **browser.py** — no CI args in headed mode (--no-sandbox etc.)
 - **agent.py** — headless, slow_mo, on_log params on run()
 - **agent.py** — emit() helper sends live events to callback
 - **agent.py** — 5 event types: start, step, step_done, done, error
 - **main.py** — POST /test-stream SSE endpoint
 - **main.py** — asyncio.Queue + create_task pipeline
 - **main.py** — 6-minute timeout safety net
 - **main.py** — DB save + screenshot URLs in result event

```
# backend/agent.py - emit helper inside run()
async def emit(event_type: str, **data):
    if on_log:
        try:
            await on_log({"type": event_type, **data})
        except Exception:
            pass
    await emit("start", url=url, goal=goal)
    await emit("step", step=N, action="click", target="Login", reasoning="...")
    await emit("step_done", step=N, status="passed", url=current_url)
    await emit("done", verdict="PASSED", total_steps=5, duration=12.4)
    await emit("error", message=str(exc))
```

```
# backend/main.py - SSE streaming endpoint (simplified)
@app.post("/test-stream")
async def run_test_stream(url, goal, headed="false", slow_mo=0, ...):
    async def event_generator():
        queue = asyncio.Queue()
        task = asyncio.create_task(run_agent_task(queue))
        while True:
            event = await asyncio.wait_for(queue.get(), timeout=360)
            if event is None:
                break
            yield f"data: {json.dumps(event)}"
    return StreamingResponse(event_generator(), media_type="text/event-stream")
```

```
// frontend/pages/index.js - fetch-based SSE reader
const response = await fetch(`${BACKEND}/test-stream`, { method: "POST", body: formData });
const reader = response.body.getReader();
let buffer = "";
while (true) {
    const { done, value } = await reader.read();
    if (done) break;
    buffer += decoder.decode(value, { stream: true });
    const chunks = buffer.split(" ");
    // split on SSE frame boundary
    buffer = chunks.pop();
    for (const chunk of chunks) {
        handleStreamEvent(JSON.parse(chunk.slice(6)));
    }
}
```

Event Type	When Emitted	Key Fields	Console Color
start	Browser opens, first page loaded	url, goal	Blue ▶
step	Before each AI action executes	step, action, target, value, reasoning, url	Yellow [N]
step_done	After action completes or fails	step, status (passed/failed), error, url	Green ✓ / Red X
done	Test finished (pass, fail, or max steps)	verdict, total_steps, duration	Green/Red ■
result	Final full result (after done)	data (full result object)	Sets TestResult component
error	Agent exception	message	Red ⚠

Problem: After adding the `/test-stream` endpoint, clicking Run Test in Headed mode showed "Not Found" in the error banner.

Root Cause

- Backend runs with `reload=False`
- Uvicorn loads code once at startup
- File changes are NOT picked up automatically
- Running process had old `main.py` — no `/test-stream` registered
- FastAPI returned `{"detail": "Not Found"} (404)`
- Frontend displayed `errData.detail` → "Not Found"

Fix Applied

- No code was incorrect — endpoint was properly defined
- Resolution: restart backend server
- Added specific "Not Found" check in frontend error handler
- Now shows: *"Headed mode endpoint not found — please restart the backend"*
- Verify via `http://localhost:8000/docs` (Swagger UI shows both endpoints)

```
// Before fix - generic unhelpful message
const msg = err.message?.includes("Failed to fetch") ? "Cannot connect to backend..." : err.message || "Unknown error";
// showed "Not Found" - confusing

// After fix - actionable message for 404
const msg = err.message?.includes("Failed to fetch") ? "Cannot connect to backend. Make sure you ran: cd backend && python main.py" : err.message === "Not Found" ? "Headed mode endpoint not found - please restart the backend: cd backend && python main.py" : err.message || "Unknown error";
```

How to restart: Stop the running backend (Ctrl+C) then run:

```
cd "AI assistant tool\i-web-tester\backend" && venv\Scripts\python main.py
```

Confirm both endpoints appear at **`http://localhost:8000/docs`**

File	Type	Change Summary
frontend/pages/index.js	Modified	File attach button + chips; headed toggle + slow-mo slider; execution console panel; <code>runTestStreaming()</code> ; <code>handleStreamEvent()</code> ; <code>useEffect</code> auto-scroll; <code>formatConsoleLog</code> helper; improved error messages
backend/main.py	Modified	POST <code>/test</code> switched to <code>multipart/form-data</code> ; <code>extract_file_text()</code> helper; new POST <code>/test-stream</code> SSE endpoint; <code>asyncio</code> + <code>StreamingResponse</code> imports
backend/agent.py	Modified	<code>run()</code> accepts <code>headless</code> , <code>slow_mo</code> , <code>on_log</code> params; <code>emit()</code> async helper; 5 event types emitted throughout the agentic loop
backend/browser.py	Modified	<code>slow_mo</code> param added; passed to <code>chromium.launch()</code> ; CI-only launch args removed in headed mode
backend/requirements.txt	Modified	Added <code>pdfplumber==0.11.4</code> , <code>python-docx==1.1.2</code> , <code>openpyxl==3.1.5</code>

 **Headless Mode (default)**

- User fills URL + goal + optional files
- Axios POST → **/test** (multipart)
- Backend parses files, builds effective_goal
- agent.run(headless=True, slow_mo=0)
- BrowserController launches invisible Chromium
- AI loop runs, steps saved to SQLite
- Full result JSON returned at end
- TResult component renders verdict + steps

 **Headed Mode (debug)**

- User toggles Browser Mode ON, sets slow-mo
- fetch() POST → **/test-stream** (multipart + SSE)
- Backend starts asyncio.Queue + create_task
- agent.run(headless=False, slow_mo=N, on_log=cb)
- Chromium opens visibly with slow-mo delay
- Each action emits SSE event → frontend console
- Console shows live colored log lines
- Final result event → TResult component

